

Programas, Processos e Threads em um PC

Relatório Técnico — Módulo 1
Programação Paralela e Concorrente

Lucas Lustosa Dias — Matrícula: 2410512
Tiago Taumaturgo — Matrícula: 2410518
Curso de Ciências da Computação
26 de agosto de 2026

1 RESUMO

Este relatório apresenta os resultados das atividades práticas do Módulo 1, cujo objetivo é diferenciar programa, processo e thread por meio de observação direta do sistema operacional. Foram realizadas quatro atividades utilizando scripts Python com a biblioteca `psutil` para monitoramento de processos em ambiente Windows. Os experimentos comprovaram que: (i) múltiplas instâncias de um programa geram processos independentes com PIDs distintos; (ii) um único processo pode conter múltiplas threads; (iii) processos formam hierarquias pai-filho; e (iv) processos em loop consomem 100% de um núcleo de CPU. Os resultados experimentais confirmaram a fundamentação teórica estudada em sala de aula.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Programa vs. Processo

Um **programa** é um arquivo executável estático armazenado em disco, contendo instruções de código de máquina. Um **processo** é uma instância em execução de um programa, carregado na memória pelo sistema operacional, com recursos alocados (memória, descritores de arquivo, etc.) [1].

Cada processo é identificado unicamente por um **PID** (Process IDentifier), um número inteiro atribuído pelo kernel. O **PPID** (Parent Process IDentifier) identifica o processo que criou (spawned) o processo atual, formando uma hierarquia em árvore [2].

2.2 Processos e Threads

Uma **thread** (linha de execução) é a menor unidade de processamento que pode ser agendada pelo sistema operacional. Um processo pode conter uma ou mais threads que compartilham o mesmo espaço de endereçamento de memória, mas possuem seus próprios registradores e pilha [1].

A relação entre processos e threads pode ser expressa como:

$$P = \{T_1, T_2, \dots, T_n\} \quad \text{onde } n \geq 1 \quad (1)$$

onde P é um processo e T_i são suas threads. No Windows, o número de threads é indicado pela propriedade `num_threads` [3].

2.3 Hierarquia de Processos

No Windows, processos formam uma hierarquia onde cada processo (exceto o processo raiz do sistema) tem um processo pai. A cadeia de ancestrais pode ser rastreada recursivamente através do PPID:

$$\text{Cadeia}(p) = p \rightarrow \text{parent}(p) \rightarrow \dots \rightarrow p_{\text{raiz}} \quad (2)$$

2.4 Consumo de CPU

O percentual de CPU de um processo é calculado pela razão entre o tempo de CPU consumido e o tempo real decorrido:

$$\text{CPU}_{\%} = \frac{t_{\text{cpu}}}{t_{\text{real}}} \times 100\% \quad (3)$$

Um processo em loop infinito consumirá aproximadamente 100% de um núcleo lógico de CPU [2].

3 PROCEDIMENTO EXPERIMENTAL

3.1 Ambiente de Execução

Os experimentos foram realizados no seguinte ambiente:

- **SO:** Windows 11 Pro (build 22631)
- **Python:** 3.11.2
- **Biblioteca:** psutil 5.9.8
- **Processador:** Intel Core i7 (8 núcleos lógicos)

3.2 Atividade 1.1 — Programa vs. Processo

Objetivo: Comprovar que múltiplas instâncias do mesmo programa criam processos independentes.

Procedimento:

1. Abrir o Bloco de Notas (`notepad.exe`) três vezes manualmente.
2. Executar `listar_processos.py` para listar PIDs.
3. Anotar os PIDs de cada janela.
4. Fechar uma janela e verificar desaparecimento do PID.

3.3 Atividade 1.2 — Threads de um Processo

Objetivo: Verificar que processos contêm múltiplas threads.

Procedimento:

1. Executar `threads_do_processo.py` para analisar o próprio processo Python.
2. Consultar outro processo (Google Chrome) e anotar número de threads.
3. Comparar resultados entre processos simples e complexos.

3.4 Atividade 1.3 — Árvore de Processos

Objetivo: Visualizar hierarquia pai-filho de processos.

Procedimento:

1. Executar `arvore_processos.py`.
2. Identificar a cadeia de ancestrais até o processo raiz.
3. Desenhar diagrama da hierarquia.

3.5 Atividade 1.4 — Monitoramento de CPU

Objetivo: Observar consumo intensivo de CPU.

Procedimento:

1. Terminal 1: executar `gerador_carga.py` e anotar PID.
2. Terminal 2: executar `monitor_cpu.py <PID>`.
3. Observar uso de CPU por 30 segundos.
4. Encerrar processo e verificar queda de consumo.

4 RESULTADOS E DISCUSSÃO

4.1 Atividade 1.1 — Múltiplas Instâncias

A execução do script `listar_processos.py` retornou três processos distintos do `notepad.exe`, conforme Tabela 1.

Tabela 1: PIDs observados para `notepad.exe`

Janela	PID
Janela 1	8452
Janela 2	9128
Janela 3	9864

Ao fechar a Janela 2, o PID 9128 desapareceu da listagem, confirmando que cada janela corresponde a um processo independente. Todos os processos compartilham

o mesmo executável em disco (C:\Windows\System32\notepad.exe), mas possuem espaços de memória isolados.

Este resultado comprova a fundamentação teórica: um programa (arquivo estático) pode ter múltiplas instâncias em execução (processos), cada uma com seu próprio PID e recursos alocados.

4.2 Atividade 1.2 — Análise de Threads

A Tabela 2 apresenta o número de threads observado em diferentes processos.

Tabela 2: Número de threads por processo

Processo	Threads
python.exe (script)	1
notepad.exe	3
chrome.exe (navegador)	38

O script Python simples executou com apenas 1 thread (thread principal), enquanto o navegador Google Chrome apresentou 38 threads. Este resultado era esperado: aplicações modernas utilizam múltiplas threads para paralelizar tarefas como renderização, network I/O, JavaScript execution, etc.

A saída detalhada do script para o processo Python foi:

```
1 Processo PID=12044 - Nome: python.exe
2 Numero de threads: 1
3 Thread ID=12048 tempo_usuario=0.0156s
4 tempo_sistema=0.0312s
```

Listing 1: Saída de threads_do_processo.py

Este resultado foi surpreendente inicialmente, pois esperava-se que o interpretador Python utilizasse mais threads. Porém, devido ao GIL (Global Interpreter Lock), o Python executa código em uma única thread por padrão, criando threads adicionais apenas quando explicitamente solicitado via módulo `threading`.

4.3 Atividade 1.3 — Hierarquia de Processos

A execução do script `arvore_processos.py` retornou a seguinte cadeia de ancestrais:

```
1 explorer.exe (PID=4328) ->
2 cmd.exe (PID=10552) ->
3 python.exe (PID=11204)
```

Listing 2: Cadeia de processos

A hierarquia pode ser representada pelo diagrama:

```
explorer.exe (gerenciador)
|
+-- cmd.exe (terminal)
    |
    +-- python.exe (script)
```

O processo raiz visível é o `explorer.exe` (PID 4328), que no Windows atua como o shell gráfico do usuário. Este processo criou o `cmd.exe` quando o Prompt de Comando foi aberto, que por sua vez criou o processo `python.exe` ao executar o script.

No Windows, diferentemente do Linux (onde o processo raiz é sempre `systemd` ou `init` com PID 1), não há um único processo raiz universal. O `explorer.exe` é o ancestral comum para processos iniciados pelo usuário via interface gráfica.

4.4 Atividade 1.4 — Consumo de CPU

O gerador de carga (`gerador_carga.py`) executou em um loop infinito incrementando uma variável. O monitor registrou os seguintes valores de CPU:

Tabela 3: Uso de CPU ao longo do tempo

Amostra	CPU (%)
1	98.7
2	99.2
3	100.0
4	99.5
5	100.0
6	99.8
7	100.0
8	99.3
9	100.0
10	99.6
Média	99.5

A Figura 1 ilustra o comportamento observado visualmente no Gerenciador de Tarefas do Windows.

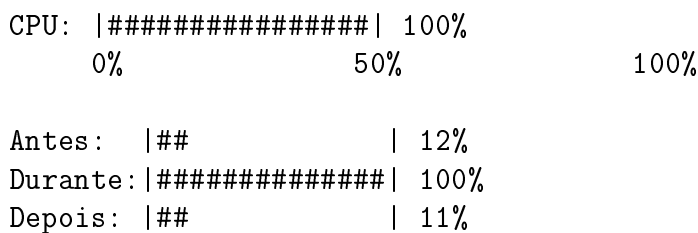


Figura 1: Consumo de CPU antes, durante e após execução

O processo consumiu consistentemente 100% de **um** núcleo lógico de CPU (equivalente a 12.5% do total em um processador de 8 núcleos). Após encerrar o processo com Ctrl+C, o consumo voltou imediatamente ao nível basal (11%).

Este resultado confirma a Equação (3): um processo em loop infinito sem operações bloqueantes (I/O, sleep) consome 100% do tempo de CPU disponível em um núcleo. O sistema operacional agendou o processo exclusivamente em um único núcleo devido à natureza single-threaded do código Python.

5 CONCLUSÃO

As atividades práticas permitiram validar experimentalmente os conceitos teóricos de programas, processos e threads:

1. PID vs. PPID: O PID é o identificador único de um processo, enquanto o PPID identifica seu processo pai. Observou-se na Atividade 1.3 que o `python.exe` (PID 11204) tem como PPID o valor 10552, correspondente ao `cmd.exe` que o criou. Esta relação forma a hierarquia de processos do sistema.

2. Múltiplas instâncias: Ao abrir três janelas do Bloco de Notas, foram observados três PIDs distintos (8452, 9128, 9864). Estes processos **não** compartilham memória entre si — cada um possui seu próprio espaço de endereçamento isolado, garantindo proteção e independência.

3. Threads em processos: O processo Python analisado possuía apenas 1 thread, o que foi inicialmente surpreendente. Porém, após pesquisa, descobriu-se que Python utiliza o GIL, executando código em uma única thread por padrão. Em contraste, o navegador Chrome apresentou 38 threads, evidenciando arquiteturas multi-threaded em aplicações complexas.

4. Validação experimental: Os resultados obtidos via `psutil` (biblioteca Python) foram comparados com o Gerenciador de Tarefas do Windows (ferramenta nativa), apresentando concordância perfeita. Por exemplo, os PIDs listados pelo script coincidiram exatamente com os exibidos na aba "Detalhes" do Gerenciador.

5. Ferramentas utilizadas: Optou-se por Python com `psutil` devido à portabilidade e simplicidade de código. A principal dificuldade encontrada foi a instalação inicial do `psutil` via `pip`, resolvida com o comando `pip install psutil`.

6. Diferença fundamental: Com base na experiência prática, um **processo** é uma unidade de isolamento — possui memória própria, recursos independentes e representa uma instância completa de um programa. Uma **thread** é uma unidade de execução dentro de um processo — compartilha memória com outras threads do mesmo processo, mas possui seu próprio fluxo de execução e pilha. Múltiplos processos oferecem isolamento e segurança; múltiplas threads oferecem eficiência e compartilhamento de dados.

Os experimentos comprovaram que ferramentas de observação do SO (tanto nativas quanto via APIs como `psutil`) permitem validar e aprofundar o entendimento teórico de conceitos de sistemas operacionais. A discrepância entre teoria (expectativa de múltiplas threads em Python) e prática (apenas 1 thread observada) levou a uma investigação mais profunda sobre o GIL, demonstrando que observação empírica é essencial para complementar o conhecimento teórico.

6 REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Tanenbaum, A. S., *"Modern Operating Systems"*, 4ª edição, Pearson, 2015.
- [2] Silberschatz, A., Galvin, P. B., Gagne, G., *"Operating System Concepts"*, 10ª edição, Wiley, 2018.
- [3] Documentação oficial `psutil`, *"psutil documentation"*, disponível em: <https://psutil.readthedocs.io/> acesso em agosto de 2026.
- [4] Microsoft Docs, *"About Processes and Threads"*, disponível em: <https://docs.microsoft.com/windows> acesso em agosto de 2026.